TP 1: lire sa machine

Objectif: savoir ce qu'il y a dans votre ordinateur, et calculer sa puissance de calcul crête. Ce chiffre servira de référence aux deux TP suivants. Durée: 1 h. Rendu: le fichier `machine.md` complété. test

1. Inventaire

Lancez le script fourni, qui appelle les bonnes commandes selon le système:

```
sh machine.sh > inventaire.txt
```

Sous Linux, installez `hwloc` pour la commande `lstopo`, qui dessine la topologie (`lstopo topologie.png`). Sous macOS, `sysctl hw machdep.cpu` donne l'essentiel; sous WSL2, `lscpu` fonctionne mais ne voit qu'une machine virtuelle.

Remplissez `machine.md` avec:

Question	Où regarder
Modèle de processeur, architecture (x86-64 ou AArch64)	`lscpu`, `sysctl machdep.cpu.brand_string`
Nombre de sockets, de cœurs physiques, de threads (SMT)	`lscpu`, `sysctl hw.physicalcpu hw.logicalcpu`
Cœurs performance et efficacité, le cas échéant (Apple M, Intel 12e gén. et plus)	`sysctl hw.perflevel0 hw.perflevel1`, `lscpu -e`
Fréquence de base et fréquence maximale	`lscpu`, `/proc/cpuinfo`, documentation du constructeur
Taille des caches L1 données, L2, L3 et taille de ligne	`lscpu`, `getconf -a | grep CACHE`, `sysctl hw.cachelinesize hw.l1dcachesize hw.l2cachesize`
Caches privés par cœur ou partagés	`lstopo`, documentation
Extensions SIMD: SSE4, AVX2, AVX-512, NEON, SVE, et leur largeur en bits	`lscpu` (flags), `sysctl hw.optional`
Mémoire vive: capacité, type (DDR4, DDR5, LPDDR5), canaux	`free -g`, `sudo dmidecode -t memory`, `system_profiler SPMemoryDataType`
Le processeur a-t-il des unités FMA (multiplication-addition fusionnée)?	flags `fma`, ou `asimd` sur AArch64

2. Puissance crête

La puissance crête en flop/s (opérations flottantes par seconde) d'un cœur est, en double précision:

```
fréquence × (largeur SIMD en bits / 64) × unités SIMD par cœur × (2 si FMA)
```

Multipliez par le nombre de cœurs. Le script `peak.py` fait le calcul:

```
python3 peak.py --cores 8 --ghz 3.5 --simd-bits 256 --units 2 --fma
```

Questions:

1. Quelle est la crête de votre machine en Gflop/s, en double et en simple précision? Pour les cœurs efficacité, faites le calcul séparément.
2. Quelle fraction de cette crête un programme Python ordinaire peut-il atteindre, sachant qu'il n'utilise ni SIMD, ni plusieurs cœurs, et qu'une opération Python coûte de l'ordre de 50 à 100 ns?
3. Quelle est la bande passante théorique de votre mémoire vive, en Go/s (fréquence effective × largeur du bus × canaux)? Combien de flop votre machine peut-elle faire pendant le temps de lecture d'un octet en mémoire? C'est l'**intensité arithmétique** minimale pour être limité par le calcul plutôt que par la mémoire; retenez-la pour le TP 3.

3. Mesure d'énergie (optionnel)

- Linux: `sudo perf stat -e power/energy-pkg/ sleep 10` donne l'énergie consommée par le processeur pendant 10 s (compteurs RAPL, Intel et AMD).
- macOS: `sudo powermetrics -i 1000 -n 10 --samplers cpu_power` donne la puissance du processeur chaque seconde.

Mesurez au repos, puis pendant `python3 -c "while True: pass"` sur un cœur. Combien de watts coûte un cœur occupé?